

BD3-LM 원본 코드 정리 & 로컬 구현 대조 검증

2026-07-16 · 원본: github.com/kuleshov-group/bd3lms (Arriola et al., "Block Diffusion: Interpolating Between Autoregressive and Diffusion Language Models", ICLR 2025) · 로컬: `models_seq/bd_models.py`, `models_seq/bd_trainer.py`, `main_bd.py`

"대조 검증 통과"의 정확한 의미. 원본 저장소를 클론하여 mask 커널의 핵심 5개 구성 요소(어텐션 마스크, 벡터화 학습, SUBS loss, first-hitting 샘플러, semi-AR 생성 루프)를 코드 수준에서 나란히 비교했고, 다섯 요소 모두 원본과 동일한 수식·알고리즘을 계산함을 확인했다는 뜻이다. 이 검증이 의미하지 않는 것: (1) 동일 가중치를 두 구현에 넣고 출력을 수치 비교하는 등가성 테스트는 수행하지 않았다 (두 코드는 vocabulary-conditioning이 달라 직접 비교가 불가능). (2) 원본의 모든 기능이 이식됐다는 뜻이 아니다 — EMA, clipped schedule 등은 의도적으로 제외됐고 아래 §4에 전부 나열했다. (3) graph(uniform) 커널은 애초에 bd3lms 이식 대상이 아니라 이 저장소 `seq_models.py` 의 loss를 블록 단위로 적용한 자체 설계다 (CLAUDE.md 지시사항).

1. BD3-LM이 풀려는 문제와 핵심 아이디어

배경. Autoregressive(AR) 모델은 우도가 높고 KV 캐시로 빠르지만 토큰을 하나씩 순차 생성해야 한다. Discrete diffusion(MDLM 등)은 병렬 생성이 가능하지만 우도가 낮고, 고정 길이 캔버스에 묶여 가변 길이 생성이 어렵다. BD3-LM은 이 둘을 **블록 단위로 보간**한다.

블록 분해. 시퀀스를 크기 B의 블록으로 나누고, 우도를 블록 단위 AR로 분해한 뒤 각 블록 내부만 masked discrete diffusion으로 모델링한다:

$\log p(x) = \sum_b \log p(x^b \mid x^{\setminus \{b\}})$ ← 블록 간: autoregressive
 $p(x^b \mid x^{\setminus \{b\}})$: 블록 내: discrete diffusion (이전 블록들을 조건으로 하는 NELBO)

- B = 10이면 순수 AR, B = 시퀀스 길이이면 순수 diffusion — **block size가 AR↔diffusion 보간 파라미터**.
- 생성은 semi-autoregressive: 블록 하나를 diffusion으로 완성해 커밋하고, 다음 노이즈 블록을 이어붙인다. 이 구조 덕분에 **가변 길이 생성**과 **블록 간 KV 캐시**가 가능해진다.

2. 원본 코드의 중요 부분 (파일·함수 단위)

2.1 벡터화 학습 — 한 번의 forward로 모든 블록 학습

`diffusion.py :: _forward_pass_diffusion`. 나이브하게 하면 블록마다 별도 forward가 필요하지만(블록 수 배 비용), BD3-LM은 노이즈 입력 x_t 와 클린 입력 x_0 를 이어붙인 $[x_t \parallel x_0]$ (길이 2n)을 한 번에 넣고, 특수 어텐션 마스크로 정보 흐름을 제어한다. 노이즈 레벨 t는 **블록마다 독립적으로 샘플**된다 (`_sample_t`: 블록 수만큼 uniform 샘플 후 `repeat_interleave`).

2.2 특수 어텐션 마스크 — 3개 서브마스크의 OR

`models/hf/modeling_bd3lm.py :: block_diff_mask`. 쿼리/키가 x_t 반쪽인지 x_0 반쪽인지(`x0_flag`)와 블록 인덱스로 판정:

M_BD (block diagonal) : 같은 반쪽 & 같은 블록 → 노이즈 블록 내부 self-attention
M_OBC (offset block causal) : x_t 쿼리 → 엄격히 이전 블록의 x_0 키 → 이전 클린 블록을 조건으로
M_BC (block causal) : x_0 쿼리 → 같거나 이전 블록의 x_0 키 → 클린 표현(캐시용) 갱신
`mask = M_BD | M_OBC | M_BC`

2.3 Loss — MDLM식 NELBO + SUBS 파라미터화

`diffusion.py :: _subs_parameterization, _loss`. 디노이저는 $\hat{x}_0$ -예측(logits over vocab). SUBS는 (a) **MASK 토큰의 확률을 0으로 강제** (`logits[:, :, mask_index] = -inf`), (b) **이미 공개된(unmasked) 위치는 자기 자신으로 carry-over** (해당 위치 log-prob = 0 → loss 기여 없음). log-linear 스케줄에서 NELBO는 다음의 단순한 가중 CE가 된다:

$$L = E_{t \sim U[\epsilon, 1]} [-(1/t) \cdot \sum_{\text{masked } \ell} \log p_{\theta}(x_{0^\ell} \mid x_t)] \quad (t \text{는 블록별 독립})$$

2.4 샘플링 — semi-AR 루프 + first-hitting 샘플러

`diffusion.py :: _semi_ar_sampler, _ddpm_caching_update`. 바깥 루프는 블록을 하나씩 이어붙이고, 안쪽 루프는 현재 블록을 디노이즈. first-hitting 샘플러(Zheng et al. 2025의 등가·고속 샘플러)는 "다음 토큰이 공개되는 시각"으로 바로 점프한다:

$u \sim U(0, 1)$; $t \leftarrow t \cdot u^{(1/\text{num_masked})}$ ← first-hitting 시간 갱신
 $p_{x0} = \text{softmax}(\text{디노이저}(x))$; masked 위치 중 균등하게 1개를 골라 p_{x0} 에서 샘플해 공개

→ 마스크된 토큰 1개당 forward 1번, 블록당 정확히 B번의 공개로 종료. 그 외 nucleus(top-p) 샘플링, 블록 완성 시 KV 캐시 저장, EOS/entropy 기반 정지 조건 (`_check_stop_conds`)이 붙는다.

2.5 분산 감소 — BD3-LM 논문의 핵심 학습 기법

- **Clipped noise schedule** (`config.algo.var_min=True` , 기본 커짐): t 를 $[\epsilon_{\min}, \epsilon_{\max}] \subset [0,1]$ 구간에서만 샘플. 학습 중 검증 loss의 **분산**이 최소가 되는 구간을 주기적으로 탐색(`_clipped_schedule_search` , delta 0.05)해 갱신한다. 논문에서 gradient 분산을 크게 줄여 PPL을 개선한 주요 기법이며, **작은 block size**일수록 효과가 큼.
- **Antithetic t 샘플링**: 배치·블록에 걸쳐 t 를 층화(stratify)해 추정 분산 감소.
- **EMA** (0.9999): 가중치 지수이동평균을 평가·생성에 사용.

2.6 백본 — DDiT (rotary + adaLN)

`modeling_bd3lm.py :: DDiTBlock` . Pre-LN transformer + **rotary 위치 임베딩**, 노이즈 레벨 σ 를 `TimestepEmbedder` 로 임베딩해 **adaLN**(zero-init, 블록당 6개 modulation: shift/scale/gate $\times$ attn/MLP)으로 주입. 단, masked diffusion에서는 마스크 토큰 자체가 노이즈 레벨을 드러내므로 **시간 조건은 사실상 불필요**(논문 명시)하고, conditioning 벡터 c 는 시퀀스당 1개로 브로드캐스트된다.

3. 구성 요소별 대조표 (mask 커널 = BD3-LM 이식 대상)

구성 요소	원본 (bd3lms)	로컬 (bd_models.py)	판정
학습 어텐션 마스크	<code>block_diff_mask</code> : M_BD M_OBC M_BC (flex-attention mask_mod)	<code>get_block_diff_mask</code> (L61): 동일 3개 서브마스크의 OR, boolean 텐서로 실체화 + 캐시	일치 — 세 서브마스크의 판정식이 문자 그대로 동일
벡터화 학습	$[x_t \parallel x_0]$ concat, 블록별 t ~ U 후 repeat_interleave	<code>_forward_mask</code> (L416): 동일 concat, <code>_sample_t_blocks</code> → repeat_interleave	일치
Loss (NELBO)	loss_scale = $-1/t$ (log-linear), SUBS carry-over로 masked 위치만 기여, 유효 토큰 수로 나눔	$-(1/t) \cdot \log p_\theta(x_0)$ 를 masked 위치에만 명시적으로 적용, loss_mask 합으로 나눔	일치 — carry-over(원본) vs 명시적 마스킹(로컬)은 동일한 값의 두 구현
SUBS 제약	logits[MASK] = $-\infty$; unmasked는 one-hot 고정	logits[...], MASK] = $-1e9$; unmasked 위치는 loss·샘플링 모두에서 갱신 제외	일치 (동작 기준)
First-hitting 샘플러	$t \leftarrow t \cdot u^{1/\text{num_masked}}$ (u는 배치 공통 스칼라); 블록 전체를 샘플 후 균등 선택한 masked 1개만 반영	<code>_denoise_block_mask</code> (L689): 동일 시간 갱신 (u는 샘플별); masked 1개를 균등 선택 후 그 위치만 p_x0에서 샘플	일치 — 커밋되는 토큰의 분포 동일. u가 샘플별인 것은 배치 내 독립성을 높일 뿐
Semi-AR 생성	<code>_semi_ar_sampler</code> : 블록 append → 디노이즈 → (EOS/entropy 정지)	<code>plan</code> (L570): 블록 append → 디노이즈 → dst(hit)/END(miss) 스캔 정지	구조 일치 (정지 규칙은 과제 특성상 대체, §4-A)
시간 조건	masked diffusion엔 사실상 불필요 (마스크가 자기 식별)	mask 커널 기본 off (<code>bd_time_cond</code> 로 옵션), graph 커널은 강제 on	일치 (원본 논문의 논거를 그대로 따름)

4. 로컬 구현이 달라진 부분 — 전체 목록

A. 도메인 적응 (언어 모델링 → OD 경로 계획)

항목	내용
Vocabulary / 상태	BPE 서브워드 → 도로 그래프 정점 id (V=1,390) + <code>END</code> (=V), <code>PAD</code> (=V+1), <code>MASK</code> (=V+2). 정점 임베딩은 node2vec으로 초기화.
캔버스 / 길이 스펙	<code>[dst, ori, v1, ..., v_L=dst, END...END PAD...PAD]</code> — 경로 최종 토큰이 속한 블록의 나머지는 END로 채워 학습 대상에 포함, 이후 블록은 PAD로 loss에서 제외 (<code>build_canvas</code> , smoke test로 경계 케이스 검증). 원본에는 없는 프로젝트 고유 설계.
OD 조건화	2채널: (1) dst-ori를 캔버스 앞 2칸에 클린 in-context 토큰으로 고정, (2) <code>od_mlp([emb(ori); emb(dst)])</code> 를 adaLN conditioning 벡터에 더함. 조건 dropout 0.1 + 학습된 null_cond → 무조건부 생성·CFG 지원 (기본 w=1, CFG off). 원본은 무조건부 생성만.
정지 규칙	원본의 EOS/entropy/likelihood 정지(<code>_check_stop_conds</code>) 대신, 커밋된 블록을 스캔해 <code>dst</code> 등장 → <code>hit</code> (성공 정지), <code>END/PAD/MASK</code> 등장 → <code>miss</code> . 길이 모델 없음 — 길이는 생성 중 창발.
후처리 (평가용)	miss 경로에 <code>refine_nx</code> (shortest-path 패치) / <code>refine_es</code> (<code>eval_shortest.refine()</code>) 변형을 추가 채점. 이번 실험은 raw(후처리 없음)만 리포트.

B. 구조적 차이 (동작은 유사하나 구현이 다른 것)

항목	원본	로컬
위치 인코딩	Rotary (상대적)	학습형 절대 위치 임베딩 (<code>pos = arange(S) % n</code> — x_t/x_0 반쪽이 위치 공유)
Conditioning 입도	c는 시퀀스당 1개 (σ 임베딩) → 브로드캐스트	c는 토큰당 (블록별 t + OD 임베딩) — 블록마다 다른 노이즈 레벨과 OD 채널을 함께 주입하기 위함
어텐션 커널	FlexAttention + torch.compile (torch 2.x)	수동 matmul + masked_fill (torch 1.12 호환; 수학적으로 동일)
어휘 헤드	weight-tying 없음, DDitFinalLayer(adaLN)	동일 구조 (final adaLN + zero-init head)

C. 원본에 있으나 로컬에서 제거된 기능

기능	원본에서의 역할	영향 평가
----	----------	-------

Clipped noise schedule (var_min 탐색)	t 샘플 구간 $[\epsilon_{\min}, \epsilon_{\max}]$ 을 검증 loss 분산 최소 기준으로 적응 탐색. 논문의 주요 기여, 작은 블록일수록 효과 큼	로컬은 $t \in [1e-3, 1]$ 고정. block size sweep 해석 시 유일하게 주의할 차이 — 작은 블록(1, 2)의 성능이 원본 대비 과소평가될 수 있음. 단 sweep 내부(블록 간) 비교는 전 구간 동일 조건이므로 공정
EMA (0.9999)	평가·생성 시 가중치 이동평균 사용	미구현. 기존 blk4 기준선(hit 0.952)도 EMA 없이 달성 — 이번 sweep도 동일 조건이라 내부 비교 공정
Antithetic t 샘플링	t 층화로 gradient 분산 감소	의도적 제거 (모듈 docstring에 명시). loss 추정 노이즈 소폭 증가
KV 캐시 / nucleus 샘플링	생성 속도 / 생성 품질-다양성 트레이드오프	속도·샘플링 온도 관련 기능 — 정확도 평가에는 영향 없음 (full softmax 샘플링 = temp 1.0)
고정 스텝 마스크 샘플러	first-hitting의 대안 (T스텝 이산화)	first-hitting만 유지 (등가·고속이므로)

D. 로컬에만 있는 것 — graph(uniform-state) 커널

bd3lms에는 없는 커널로, 이식이 아니라 결합이다: BD3-LM의 블록 구조(어텐션 마스크, $[x_t \parallel x_0]$ 학습, semi-AR 생성)는 그대로 두고, 블록 내부의 노이즈·loss만 이 저장소 `seq_models.py` (Destroyer/Restorer)의 uniform-state CTMC diffusion으로 교체했다 (block-diffusion-loss.pdf, CLAUDE.md 지시).

- 노이즈: $Q_t = \exp((A' - D')\beta_t)$ — 그래프 인접행렬 기반 CTMC, 상태공간은 $\langle \text{end} \rangle$ 상태를 추가한 $V+1$ (가상 $\langle \text{end} \rangle$ 노드 degree = `bd_eos_deg`).
- Loss: D3PM ELBO = $KL[q(x_{t-1}|x_t, x_0) \parallel p_\theta] + CE + \text{connectivity 정규화}$ (Restorer.forward와 동일 수식, 블록·loss_mask 단위 적용).
- 생성: 블록당 `max_T(100)`스텝 역과정 + MC posterior 근사, 마지막에 인접행렬 제약 디코드.
- 시간 조건 강제 on (uniform 노이즈는 x_t 만으로 노이즈 레벨 식별 불가).

5. 요약

- mask 커널은 BD3-LM의 수학(마스크·loss·샘플러)을 정확히 따른다 — 위 §3의 다섯 요소 모두 코드 수준 대조로 확인.
- 차이는 세 종류로 정리된다: (A) 과제 적응(OD 조건화·길이 스펙·정지 규칙 — 프로젝트 스펙에 의한 필수 변경), (B·C) 학습 편의 기능 제거(EMA·clipped schedule·antithetic 등 — 정확도에 2차적 영향, sweep 내부 비교에는 영향 없음), (D) graph 커널 추가(bd3lms 외부의 자체 설계).
- 결과 해석 시 유의점 한 가지: clipped schedule 부재는 작은 block size의 절대 성능을 원본 논문 대비 불리하게 만들 수 있다. block size 간 경향 비교는 유효하 되, "blk1이 논문의 AR 수준"이라는 식의 절대 비교는 피할 것.